Modelo Relacional UML da Aplicação UnB RU

Grupo 8 Isabela de Souza Kassio Gandara de Souza
Adriano Lopes
Natália Leal Pedro de Paula

Introdução

O **UnB RU** é um sistema, desenvolvido em *Ruby on Rails*, para o controle de acesso aos restaurantes universitários da UnB. O presente relatório descreve o modelo relacional UML do sistema, reunindo as funcionalidades de todos os integrantes do Grupo 8, e apresenta a investigação técnica sobre a transformação das telas do protótipo do Figma em views Rails.

As funcionalidades foram divididas entre os integrantes conforme as issues do repositório da disciplina. As issues não têm responsável atribuído no GitHub, e a divisão foi identificada pelos *pull requests* e pelos *commits* do grupo:

- **Adriano Lopes**: Cadastro de Usuário (Issue #1) e Relatório Global de Acessos (Issue #9).
- **Isabela de Souza**: Configuração de Ponto de Acesso (Issue #5) e Validação de Entrada (Issue #6).
- **Kassio Gandara de Souza**: Extrato e Histórico (Issue #3), Histórico Local da Catraca (Issue #7), Gestão de Restaurantes e Catracas (Issue #8) e Exportação em PDF do Relatório de Operação (Issue #12).
- **Natália Leal**: Recarga de Saldo (Issue #2) e Cardápio da Refeição Atual (Issue #10).
- **Pedro de Paula**: Autenticação do Funcionário (Issue #4) e Exportação em PDF do Histórico do Estudante (Issue #11).

Entidades Principais

- **Usuario**: **id** é a chave primária. A matrícula é única e o perfil define se o usuário é estudante, funcionário ou administrador. O saldo é debitado a cada entrada.
- **Restaurante**: **id** é a chave primária. Possui nome único, capacidade de 1 a 5000 refeições por hora, horário de abertura anterior ao de fechamento e status (ativo ou inativo).

- **Catraca**: **id** é a chave primária. **restauranteId** é chave estrangeira obrigatória para Restaurante e **responsavelId** é chave estrangeira opcional para Usuario. O código é único dentro do restaurante e o status pode ser ativa, inativa ou em manutenção. Uma catraca com acessos ou responsável vinculados não pode ser apagada do banco (`ON DELETE RESTRICT`), e a exclusão é lógica, com status *Inativa*.
- **Acesso**: **id** é a chave primária. **catracaId** é chave estrangeira para Catraca e **usuarioId** é chave estrangeira para Usuario, vazia quando a matrícula não está cadastrada. Registra data e hora, resultado, motivo da negativa e valor cobrado, e é preservado para auditoria.
- **Recarga**: **id** é a chave primária e **usuarioId** é chave estrangeira para Usuario. Registra valor, método (PIX ou cartão de crédito), status e data e hora.
- **Cardapio**: **id** é a chave primária. Representa a refeição de uma data.
- **ItemCardapio**: **id** é a chave primária e **cardapioId** é chave estrangeira para Cardapio. Cada item possui tipo (padrão, ovolactovegetariano ou vegetariano estrito) e descrição.

Modelo Relacional UML

Abaixo está o modelo relacional UML que descreve a estrutura de dados da aplicação **UnB RU**:

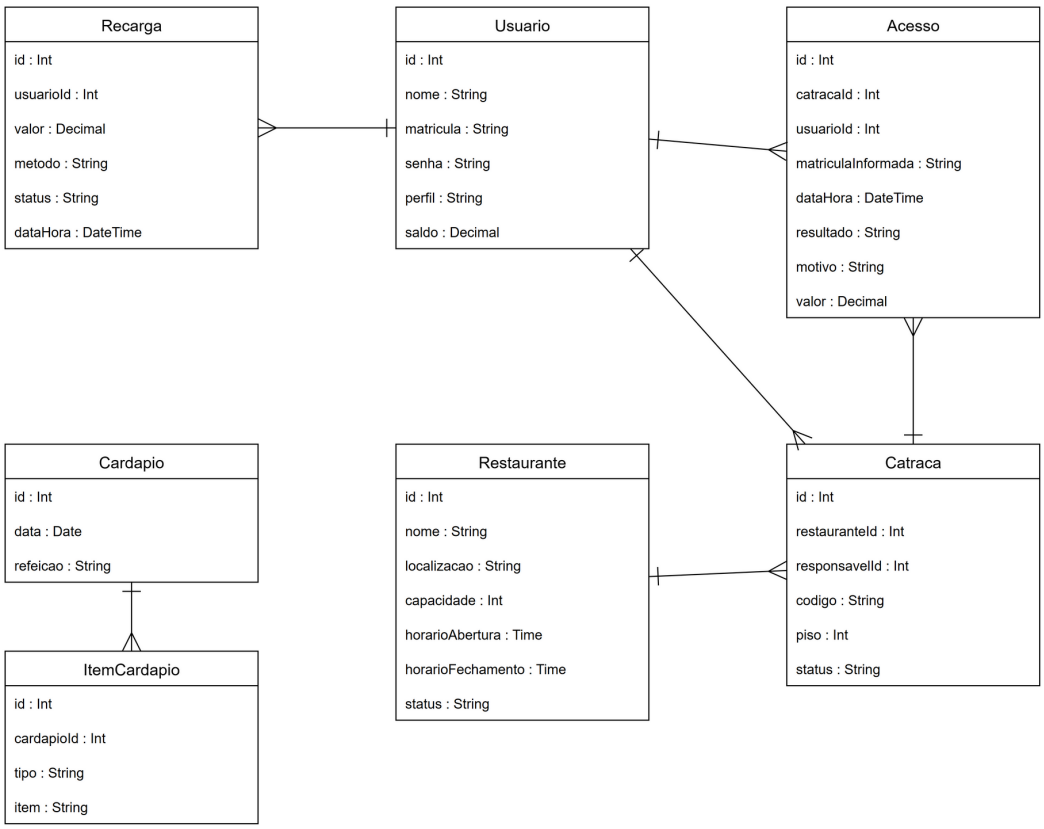


Figura 1: Modelo Relacional UML da aplicação UnB RU.

Investigação Técnica (Spike)

O spike analisou as 12 telas do protótipo no Figma, exportadas em PDF e SVG, e definiu como transformá-las em views Rails. As telas se organizam em três áreas, e cada área corresponde a um *layout*: o Portal do Estudante (`estudante`), a Aplicação de Controle (`operador`) e a Administração (`administracao`). As rotas são agrupadas por perfil, cada controller exige o perfil correspondente por meio de um `before_action`, e os componentes repetidos, como cartões de resumo, filtros, tabelas, selos de status e botões de exportação, ficam em *partials* compartilhados. Os valores de exemplo do Figma (como R$ 72,00 ou 4.820 acessos) correspondem a agregações (`sum` e `count`) sobre os modelos. Cada tela é descrita a seguir, com a issue correspondente.

1. Início e Carteira (Issue #10)

- **Elementos**: saudação com nome, matrícula e curso; cartão de saldo (R$ 18,00, equivalente a 4 refeições, tarifa única de R$ 4,50); estado dos três turnos do dia (encerrado, atual e próximo); cardápio do almoço com as opções padrão, ovolactovegetariana e vegetariana restrita, cada uma com prato principal, guarnições, salada, sobremesa e bebida; aviso de FastPass ativo; botão *Recarregar Saldo*.
- **Rails**: rota `GET /`, `InicioController#show`, view `inicio/show.html.erb` com os partials `_cartao_saldo`, `_turnos` e `_opcao_cardapio`.
- **Dados e regras**: o método `Cardapio.atual` escolhe a refeição pelo horário do servidor e carrega os `ItemCardapio` agrupados por tipo. Sem cardápio cadastrado, a tela exibe que o cardápio ainda não foi disponibilizado, e fora do horário de serviço informa o próximo turno.

2. Recarga de Saldo (Issue #2)

- **Elementos**: pacotes de 2, 5, 10 e 20 refeições (R$ 9,00, R$ 22,50, R$ 45,00 e R$ 90,00), campo para outra quantidade, escolha entre PIX e cartão de crédito, resumo com o novo saldo, chave PIX com o botão *Copiar Código* e botão *Confirmar*.
- **Rails**: rotas `GET /recarga` e `POST /recarga`, `RecargasController#new` e `#create`, formulário `form_with model: @recarga`. Um controller Stimulus calcula o total (quantidade × R$ 4,50) e copia o código PIX.
- **Dados e regras**: valores negativos ou nulos são rejeitados. A recarga por PIX nasce *pendente* e passa a *aprovada* na confirmação, e a por cartão sem limite fica *recusada*, sem alterar o saldo. O crédito do saldo ocorre dentro de uma transação (`ActiveRecord::Base.transaction`).

3. Extrato de Refeições (Issues #3 e #11)

- **Elementos**: três cartões de resumo (total gasto no mês, refeições realizadas e total recarregado); filtros de período, tipo de refeição, restaurante e forma de acesso, com o link *Limpar Filtros*; tabela de lançamentos com data e hora, refeição ou evento, restaurante e catraca, método de acesso, valor (débitos em vermelho e recargas em

verde) e comprovante; paginação com itens por página; botões *Exportar PDF* e *Imprimir*.

- **Rails**: rota `GET /extrato`, `ExtratosController#index`, partials `_resumo`, `_filtros`, `_tabela` e `_vazio`. A exportação é o mesmo caminho com a extensão `.pdf`, e a impressão utiliza `window.print()` com CSS `@media print`.
- **Dados e regras**: os lançamentos reúnem `current_user.recargas` e `current_user.acessos` em ordem decrescente de data, convertidos por um objeto de apresentação. As datas seguem dd/mm/aaaa (`Date.strptime`), e data inicial posterior à final gera erro. Sem resultados, o partial `_vazio` exibe a mensagem de lista vazia.

4. Autenticação Operacional (Issue #4)

- **Elementos**: campo de matrícula SIAPE ou usuário, campo de senha (ambos obrigatórios), identificação da unidade e botão *Entrar no Sistema*.
- **Rails**: rotas `GET /operador/entrar` e `POST /operador/entrar`, `Operador::SessoesController#new`, `#create` e `#destroy`. A tela utiliza o layout `operador` sem a barra de navegação, que só aparece após o login.
- **Dados e regras**: o modelo `Usuario` utiliza `has_secure_password`, e o `create` grava o identificador na sessão. Credenciais incorretas ou não cadastradas exibem "Credenciais inválidas", e campos vazios exibem o alerta de preenchimento obrigatório, sempre mantendo a tela de autenticação.

5. Configuração de Ponto (Issue #5)

- **Elementos**: seleção da unidade operacional, seleção da catraca física agrupada por piso com seu estado (online ou pronta), relógio do servidor sincronizado por NTP, sessão de refeição atual, trava anti-reentrada de 120 minutos, resumo da vinculação e botão *Confirmar e Iniciar Operação*.
- **Rails**: rotas `GET /operador/ponto` e `PATCH /operador/ponto`, `Operador::PontosAcessoController#show` e `#update`, partial `_catraca_opcao` e agrupamento com `group_by(&:piso)`. O relógio é atualizado por um controller Stimulus a partir da hora enviada pelo servidor.
- **Dados e regras**: a lista contém apenas restaurantes ativos e catracas ativas do restaurante escolhido. Os identificadores escolhidos ficam na sessão. Confirmar sem selecionar exibe "Selecione um restaurante para continuar", e restaurante inativo exibe "Restaurante indisponível para operação". Estudantes recebem acesso negado.

6. Terminal de Validação (Issue #6)

- **Elementos**: horário oficial, alternância entre Modo Comum e Modo Exclusivo FastPass, leitura de QR Code, painel de resultado (acesso liberado ou bloqueado), identificação do estudante, débito e saldo restante, simulador de leituras (*Simular OK*, *Fora Janela* e *Sem Saldo*) e tabela das últimas validações com horário, matrícula, modalidade, valor e status.

- **Rails**: rotas `GET /operador/terminal` e `POST /operador/validacoes`, `Operador::TerminalController#show` e `Operador::ValidacoesController#create`. A resposta do `create` é um *Turbo Stream* que atualiza o painel de resultado e a tabela sem recarregar a página.

- **Dados e regras**: um objeto de serviço (`ValidacaoEntrada`) aplica as regras: matrícula com 9 dígitos, estudante cadastrado, catraca disponível e saldo de pelo menos R$ 4,50. Cada tentativa grava um `Acesso` liberado ou negado, com o motivo, e a liberação debita o saldo. O simulador fica restrito ao ambiente de desenvolvimento.

7. Histórico Local de Acessos (Issue #7)

- **Elementos**: título com a catraca, unidade, posto e turno; cartões do turno (giros confirmados, validações FastPass e total debitado); filtros de status e modo; botões *Atualizar Lista* e *Exportar Relatório (CSV/PDF)*; tabela com horário, matrícula, modo de validação, tarifa e status da catraca (liberado, ou bloqueado com o motivo), com as linhas bloqueadas destacadas; paginação.

- **Rails**: rota `GET /operador/historico`, `Operador::HistoricoController#index`, partials `_resumo_turno`, `_filtros`, `_tabela` e `_selo_status`. A exportação utiliza `respond_to` com os formatos `csv` e `pdf`.

- **Dados e regras**: a consulta é `Acesso.where(catraca: current_user.catracas)`, restrita às catracas do funcionário, e os filtros atuam sobre `resultado` e sobre o modo. Funcionário sem catraca vinculada vê a mensagem para procurar o administrador. Os cenários pedem também filtro por período (dd/mm/aaaa) e hora (HH:MM), ausente da tela.

8. Gestão de Restaurantes e Catracas (Issue #8)

- **Elementos**: cartões de totais (restaurantes, catracas e corredor FastPass); botões *Cadastrar Restaurante* e *Associar Nova Catraca*; lista de unidades com capacidade total em refeições por hora, catracas ativas e os botões *Editar Unidade* e *Ver Catracas*; catracas agrupadas por piso, com status.

- **Rails**: `Admin::RestaurantesController` (`index`, `new`, `create`, `edit` e `update`) e `Admin::CatracasController` (`create` e `update`), formulários `form_with` e partial `_catraca_card`. A rota de `destroy` da catraca não existe, e a inativação é uma atualização de status.

- **Dados e regras**: nome único, localização obrigatória, capacidade de 1 a 5000, abertura anterior ao fechamento e código da catraca único dentro do restaurante. Inativar um restaurante inativa as catracas ativas na mesma transação. Restaurante só é removido sem catracas vinculadas, e funcionários acessam a tela apenas para leitura.

9. Relatório Global de Operação (Issues #9 e #12)

- **Elementos**: filtros de data contábil, turno e unidade, com o indicador de catracas online; quatro cartões (refeições servidas, arrecadação global, acessos via FastPass e

pontualidade); tabela de consolidação por restaurante com acessos, FastPass, pontualidade e arrecadação bruta; métodos de liquidação (PIX e cartão); distribuição por refeição; botões *Exportar CSV* e *Exportar PDF Oficial*.

- **Rails**: rota `GET /admin/relatorio`, `Admin::RelatoriosController#show`, com a classe de consulta `RelatorioOperacao`, que recebe data, turno e unidade e serve à tela e ao PDF. A exportação utiliza `respond_to` e a gem `prawn`.
- **Dados e regras**: as agregações utilizam `group(:restaurante_id)`, `count` e `sum(:valor)` sobre `Acesso`, e `group(:metodo)` sobre `Recarga`. O PDF traz o período, a data e hora de emissão e o administrador, só é gerado quando há registros e nunca é enviado pela metade. Apenas o administrador acessa.

10. Sorteio FastPass (épico bônus)

- **Elementos**: passe do próximo turno (contemplado, número do sorteio, validade, turno, janela, catraca, tolerância de 5 minutos, código do passe e botão *Liberar vaga*); ciclo diário de refeições; inscrição no sorteio com escolha da unidade e da janela exclusiva, horário do resultado e botão *Inscrever-se no Sorteio*.
- **Rails**: `FastpassController#show` e `InscricoesFastpassController#create` e `#destroy`, com os modelos `InscricaoFastpass` e `PasseFastpass`. O sorteio é executado por uma tarefa agendada (`ActiveJob`) 60 minutos antes da abertura do turno.
- **Dados e regras**: cada estudante possui uma inscrição por turno, e a não utilização do passe sem cancelamento gera bloqueio de 24 horas em novos sorteios.

11. Turnos e Time-frames (épico bônus)

- **Elementos**: três turnos (café da manhã, almoço e jantar) com horário, duração, situação e modo FastPass; parâmetros de granularidade (15 minutos), tolerância (3 minutos) e antecedência (2 minutos); grade de lotes ativáveis por clique; opção de pontualidade rígida; botões *Salvar* e *Restaurar Padrão*.
- **Rails**: `Admin::TurnosController#index` e `#update`, modelos `Turno` e `LoteFastpass`, e um controller Stimulus que alterna o estado de cada lote.
- **Dados e regras**: a tela informa que as alterações chegam às catracas em até 60 segundos, o que corresponde a um cache da configuração com expiração de 60 segundos (`Rails.cache.fetch`).

12. Vagas e Cotas FastPass (épico bônus)

- **Elementos**: cartões de cota diária total, taxa de ocupação e capacidade segura por catraca; grade de janelas com a cota, a carga por catraca e um campo de ajuste manual; total ofertado; botões *Redefinir Padrões* e *Aplicar Cotas ao Próximo Sorteio*.
- **Rails**: `Admin::CotasFastpassController#index` e `#update`, com atualização em lote por `fields_for` e o modelo `CotaFastpass`.

- **Dados e regras**: a carga por catraca é calculada pela divisão da cota pelo número de catracas ativas, e a cota de uma janela não pode ultrapassar a capacidade segura da catraca.

Divergências em relação às features

- A recarga do Figma vende pacotes de refeições, e os cenários tratam de valores em reais.
- A autenticação do Figma aceita matrícula SIAPE ou usuário SIGAA/gov.br, e o cenário exige matrícula de 7 dígitos.
- O Terminal de Validação do Figma possui os modos Comum e FastPass, ausentes dos cenários de Validação de Entrada.
- O histórico local do Figma filtra por status e modo, e os cenários filtram por período e hora.
- O Cadastro de Usuário (Issue #1) não possui tela no Figma, e as telas de FastPass, turnos e cotas não possuem features no repositório.

Considerações Finais

O modelo reúne as entidades das funcionalidades do grupo e evidencia o vínculo entre restaurantes, catracas e acessos. O spike mostra que as 12 telas do protótipo cabem em três layouts e em um conjunto pequeno de componentes compartilhados. Permanecem a alinhar no grupo as divergências entre as telas do Figma e os cenários.